

Your Agent Escaped Without Escaping the Sandbox

The harness stays in the dev sandbox,
poisons what the CI runner trusts,
and the runner promotes that bug into the prod sandbox.

Yossi Eliaz, PhD • Incredibuild • 21 October 2026

Spoken notes generated from slides/talk.tex. Slides 1–12 are the talk. Slides 13–18 are for questions.

1. Your Agent Escaped Without Escaping the Sandbox

00:00-00:35

Say the one line. Then the three statuses.

A factory starts with a written definition of correct. A harness works in a dev sandbox to satisfy it. A verifier reads that definition. The CI runner promotes on PASS. The prod sandbox runs the bytes.

In this recording the verifier reads `expected.json` from the workspace the harness writes. No login on admin must return 401. The program returns 200. The harness writes 200 into that file. The verifier prints PASS. The runner promotes the program.

I am Yossi Eliaz. The line on the slide is the reason.

2. Today's factory

00:35-01:30

Read spec, harness, verifier, runner, prod. Then the line.

This is the factory teams are running now. A spec says what correct means. A harness writes code in a dev sandbox to match it. A verifier checks the result. A CI runner promotes on PASS. A prod sandbox runs the published bytes.

Tessl describes that stack as context, then a loop with a verifier, then a factory. The verifier is there because a written skill is advice until something checks the output.

In our recording the verifier loads `expected.json` from the dev sandbox. The harness can write that file. The rest of the talk is that edit, then the three other inputs the runner also trusts: the token, the next job's disk, and the allowed host.

3. Two files. One of them ships.

01:30-02:20

Point at print 200. Then at `expected.json`.

Two files. `handler.py` is what the prod sandbox will run. `admin:none` means `/admin` with no login. A correct app prints 401. This one prints 200 for every request. That is an open admin page.

checker.py is what the runner trusts, and it is read-only. It loads expected.json and prints PASS when the program matches that file. The harness cannot change the checker. It can change the answer key, because that file lives in the dev sandbox.

The next three slides are the other doors the factory left open: the runner token, the next job's disk, and the allowed publish host. Then we come back and flip the answer key.

4. Runner token. Inside the sandbox.

02:20-03:30

200, then 403, then this job's input still works.

The runner injected its token into the dev sandbox so the harness could read the repo. The harness sends that token.

A GET for another project's private fixture returns two hundred and the synthetic private fixture. The sandbox did not steal a credential. It used the one the workflow placed in the environment.

A token minted for this task gets four-oh-three on the other project and two hundred on its own input. Expired and wrong-audience tokens get four-oh-three too.

The lock sits on the runner: project this token for this task, this audience, this hour. A short lifetime on the runner secret is still the runner secret.

5. Next job's file. Writable.

03:30-04:40

Host file changes, then the same write is denied.

The factory mounted the next job's directory into the dev sandbox and left it writable, so jobs could share a queue. The harness writes next-job.json. Ordinary open. The host file changes.

The same write through a read-only bind is denied, and the host file stays. The harness can still write its own patch to output.

That mount table is the lock. Input read-only, output writable, the next job absent. Fork this Boat disk for another harness and the files ride along. The fork kept the disk. Choosing who may write it is a separate authority.

6. One allowed host. Wrong account.

04:40-05:50

201 on the wrong account. Then the runner's own report.

The runner allows one host so the harness can publish a report or an artifact. The hostname is the one on the workflow allow list.

The harness posts the private fixture to another account on that host. The service stores it. Two-oh-one. The receiver has the bytes. The data left through a door the runner opened.

The repair keeps the POST on the runner. The harness may name one operation. Extra fields are denied and nothing is stored. The dev sandbox has no route of its own to the service. The runner still delivers the team report.

One allowed hostname still contains many operations.

7. No login. Test says PASS.

05:50-07:50

Say no login. Then 401 becomes 200. Then PASS. Pause.

admin:none is the lab name for one request. The route is admin. The login is none. Nobody is authenticated. A correct app returns 401. This app returns 200 for every request. That is an open admin page.

The runner decides promote from a checker. The checker program is read-only. The harness tries to overwrite it. The mount denies the write. The hash matches before and after. With the honest answer key the checker prints FAIL, because 200 is not 401. The check can see the bug.

The checker loads expected.json from the dev sandbox. The harness changes the answer for no login from 401 to 200, and does the same for the other cases. The app still returns 200. Same checker. Still read-only. It prints PASS, because the app matches the new answer key.

The runner is now green. The handler it would publish still returns two hundred. We protected the grader and left the answer key with the harness. The filename is expected.json.

8. Same bytes, three stations

07:50-09:05

PASS is the harness. REJECT is the runner. ACCEPT is the real fix.

Same bad bytes. Three stations.

The honest file in the dev sandbox: FAIL. The file the harness wrote: PASS. That PASS is what the runner would promote. Cases the runner owns, outside the dev sandbox: REJECT.

Then the factory still has to ship. The corrected handler returns four-oh-one for anonymous admin and for an expired credential, four-oh-three for an ordinary user, and two hundred for the administrator and the public route. The runner accepts it.

Move the answer key to the runner. The prod sandbox can receive the real fix.

9. Prod runs the judged bytes.

09:05-10:25

Denied, published, denied. Point at the order.

Green on the runner is not yet a prod sandbox. Freeze the candidate in dev before the run. Execute those frozen bytes where the harness cannot edit them. Compare against cases the runner owns. Then one fresh approval.

Bad bytes under that approval: DENIED. The judged bytes: PUBLISHED. Those bytes again: DENIED.

Say the order. Denied, published, denied. Spend the approval before the digest check, and a broken check hides behind an already-used approval. This repository used to do that.

Publication is those exact bytes and the spent approval, one transaction, in storage the dev sandbox cannot reach. The prod sandbox runs that object. A harness that prints finished holds no approval.

10. Fork the disk. Fork the key.

10:25-11:15

Say this follows from the file. Then move.

Start the reviewer harness from a fork of this Boat disk. Start the deploy job from it too. If they all read expected.json, and the dev harness can write it, three stations share one definition of done.

The fork records the disk. It does not record who was allowed to choose the criteria. This follows from the file the harness changed. A separate attack across harnesses was not run. A workspace file needs an owner before any later sandbox treats it as policy.

11. Rewrite four lines.

11:15-12:50

Each row: the abuse fails, and the developer's ticket still ships.

Rewrite four lines in your own factory. The abuse fails. The developer's ticket still completes.

Which token does the harness find in the dev sandbox, and what else can it read? Which paths are mounted, including a disk you will fork for the next harness? Which host can it publish to, and who writes the path and the body? Which file does the runner's check open, and can the harness edit it?

The component that returns four-oh-three, a read-only denial, DENIED, or reject sits on the runner, outside the dev sandbox. After it is there, run the harness again on the real ticket.

A green runner is still three decisions. These bytes met these criteria. This approval authorizes these bytes for the prod sandbox. The record matches an anchor the harness cannot rewrite. A clean log can store a bad promote.

12. Who pressed promote?

12:50-14:00

Leave the question up. Stop by 14:00.

The developer asked for a fix. The harness wrote 200 into the proof. The runner said PASS. The prod sandbox ran that program.

Who pressed promote?

Keep the criteria on the runner. Freeze the candidate. Run those bytes where the harness cannot edit

the cases. Publish those bytes into the prod sandbox, and only those bytes. Run the four bad lines again. Run the real ticket again.

The repository has the source, the recordings from the Boat dev sandbox, and the limits. Four doors from the abstract. Four locks. Open the file your runner's check trusts. Thank you.

Questions

13. The five policy cases

Questions

Use only in questions.

The handler takes a route and an authentication label and prints an integer. The runner checks these five. The repaired handler matches all five. This is the model of the check that promotes into the prod sandbox.

14. Runner token and the dev disk

Questions

Use only in questions.

The token case is loopback HTTP. The broad fixture stands in for the runner secret. The task fixture stands in for a job-scoped token. The mount case watches the host file after an ordinary write from the dev sandbox. This job's output stays writable.

15. Allowed host, wrong account

Questions

Use only in questions.

The fixture service permits both account paths, the way one allowed deploy host holds more than one operation. The delivery record shows the bytes arrived. The repair moves the POST onto the runner and removes the dev sandbox's direct route.

16. An earlier promote stored a marker

Questions

Use only in questions.

The archived case mutates a controller-side object, keeps a stale digest, and the old gate records a publication marker. The current promote derives the digest from immutable bytes and stores those bytes. Reaching the old flaw takes access to the controller object.

17. What was recorded

Questions

Use only in questions.

The records bind their source. The dev sandbox recordings use bubblewrap on a Boat Ubuntu machine. The harness under test is a script. The prod step in the recording is the runner's publication of exact bytes, the shape of what a prod sandbox is allowed to execute. The forked-disk slide follows from that workspace file. The timings are a delivery budget.

18. Reproduce the factory

Questions

Use only in questions.

Run the fixtures on disposable Linux with bubblewrap. The deck reads those recordings. Poisoned pipeline execution is the public name for a CI job that runs an attacker-influenced file. Anthropic's reward-hacking study is prior motivation and its measurements stay with that study. Reproducing the talk needs no GitHub token.